

Topics I Enjoyed Learning in CS 6650

"Distributed systems is less about memorizing components and more about reasoning from constraints and measured behavior."

This reflection distills what I found most valuable across **class slides** (Days 1–6), **assignments** (HW1–HW6), and **foundational readings** (Lampert, Parnas, Mixu Chapters 1–2). Each section captures a concept that shifted how I think about building systems at scale.

Table of Contents

- 1. [From One Machine to Many](#)
- 2. [Concurrency and Shared Memory](#)
- 3. [Persistence and Coordination](#)
- 4. [APIs to Load Balancers](#)
- 5. [Reproducibility Through Automation](#)
- 6. [Performance and Tail Latency](#)
- 7. [Horizontal Scaling and Resilience](#)
- 8. [Readings That Changed How I Think](#)
- 9. [Interview-Ready Takeaways](#)
- 10. [Closing Reflection](#)

1) From One Machine to Many: My Evolving Mental Model

The Aha Moment

Distributed systems feel intimidating at first — there are so many moving parts, failure modes, and unfamiliar abstractions. But what clicked for me is that nearly every challenge we studied can be traced back to a handful of **recurring forces**:

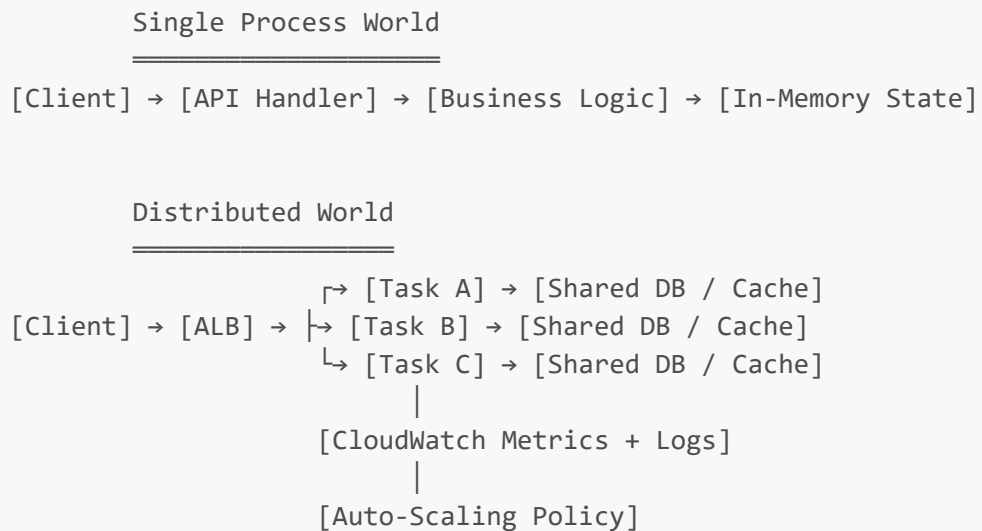
Force	What It Means	Where It Showed Up
Coordination	Getting multiple parties to agree or act in sequence	2PC, consensus, task orchestration
Contention	Multiple actors fighting over the same resource	Mutex experiments, CPU saturation under load
Partial Failure	Some parts fail while others keep running	Health checks, replica failover, network partitions
Observability	Making the invisible visible so you can reason about behavior	CloudWatch, Locust graphs, <code>-race</code> detector

The course kept reintroducing these forces at progressively larger scopes — first inside a single Go process (goroutines competing for a map), then across containers, then across services behind a load balancer, and

finally across auto-scaling clusters. Every layer of zoom changes the vocabulary but not the underlying physics.

The One Diagram I'd Draw

If I could only sketch one picture on a whiteboard, it would be this pipeline — annotated with what changes as we go from a single process to a distributed fleet:



What changes between the two worlds: state moves from local memory to a shared store, a load balancer appears to distribute requests, health checks replace assumptions about availability, and observability tooling becomes non-negotiable because you can no longer just attach a debugger.

2) Concurrency and Shared Memory: Why Correctness Is Hard

Why This Section Matters

HW3 was the assignment that made concurrency feel *real* — not as a textbook concept, but as something you could observe breaking in front of you. The key insight: **concurrency bugs are silent**. Your program might run correctly 99 times and corrupt data on the 100th. Evidence-driven testing is the only reliable defense.

The Experiments and What They Taught Me



Atomicity and Race Conditions

A simple shared counter incremented by multiple goroutines produced **surprising, nondeterministic values**. The root cause: interleaved reads and writes with no synchronization. Running with Go's `-race` flag made the invisible visible — it flagged data races that might never crash the program but can silently corrupt results.

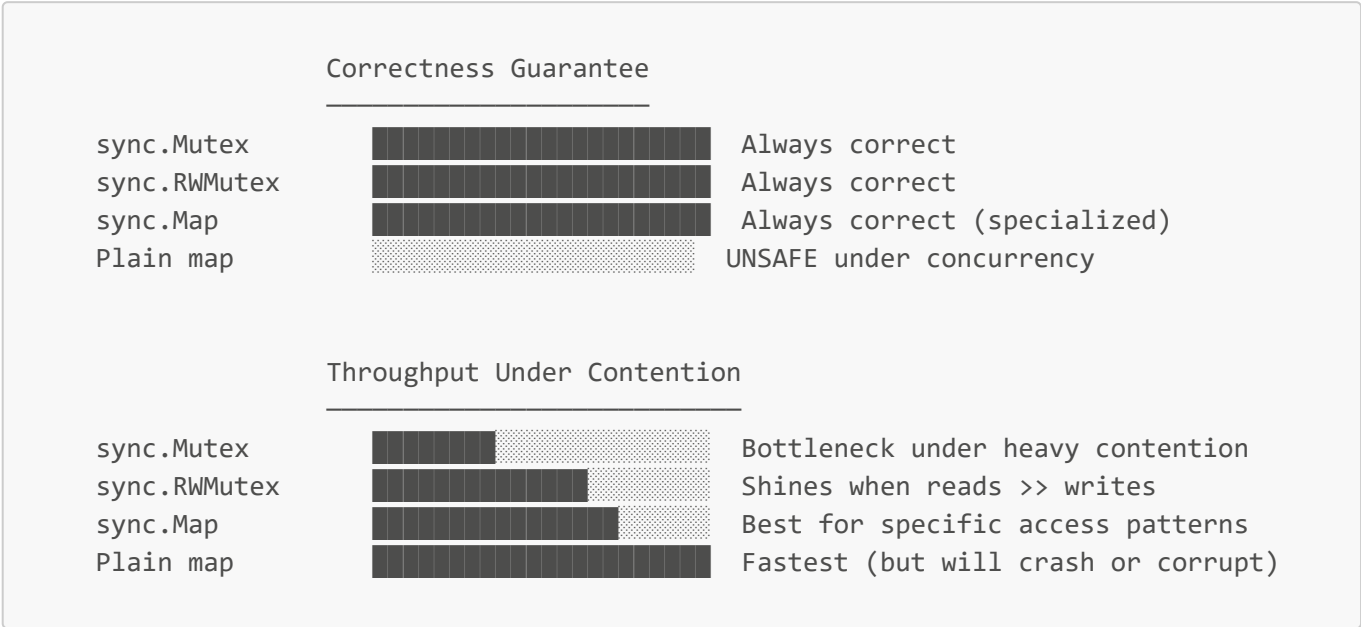
Lesson: The absence of a crash does not mean the absence of a bug.

Maps and Crashes

Concurrent writes to a plain Go `map` don't just produce wrong answers — they can **crash the entire runtime**. Go's `map` implementation explicitly detects concurrent writes and panics, which is actually a safety feature. A silent corruption would be far worse.

The Lock Spectrum: `Mutex` → `RWMutex` → `sync.Map`

This was my favorite experiment because the tradeoffs map directly to real system design decisions:

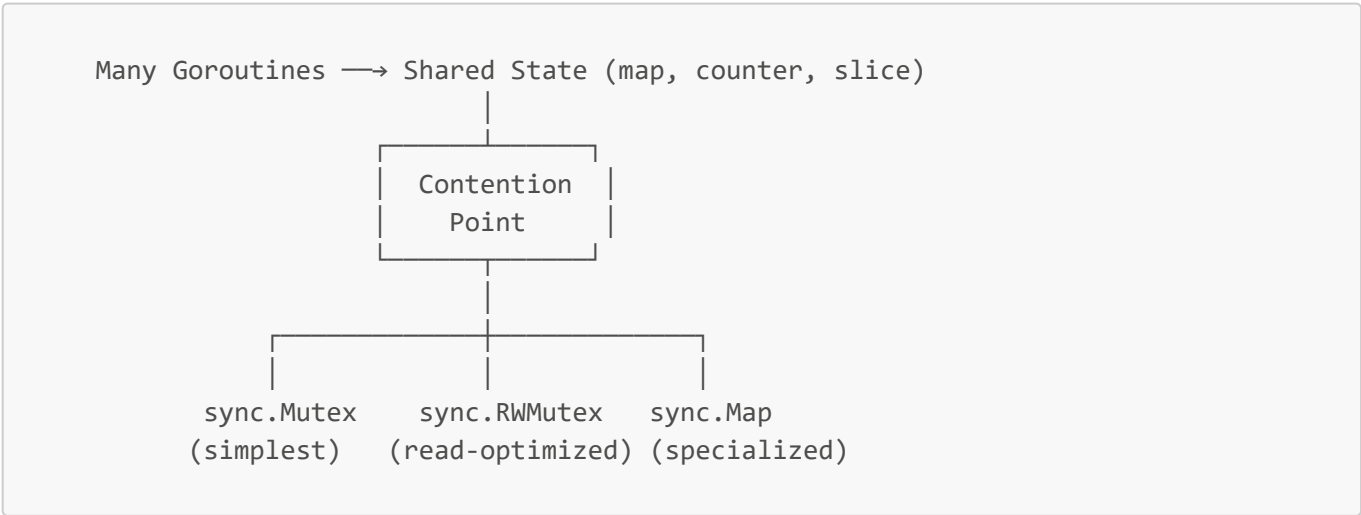


Key insight: Performance numbers only make sense when you tie them to the **read/write ratio** and **contention level** of your workload. There's no universally "best" synchronization primitive.

Context Switching Costs

Ping-pong experiments between goroutines estimated the cost of scheduling and handoff. Even though goroutines are lightweight compared to OS threads, switching still has a measurable cost that compounds under high concurrency.

Mental Model



The Conceptual Leap

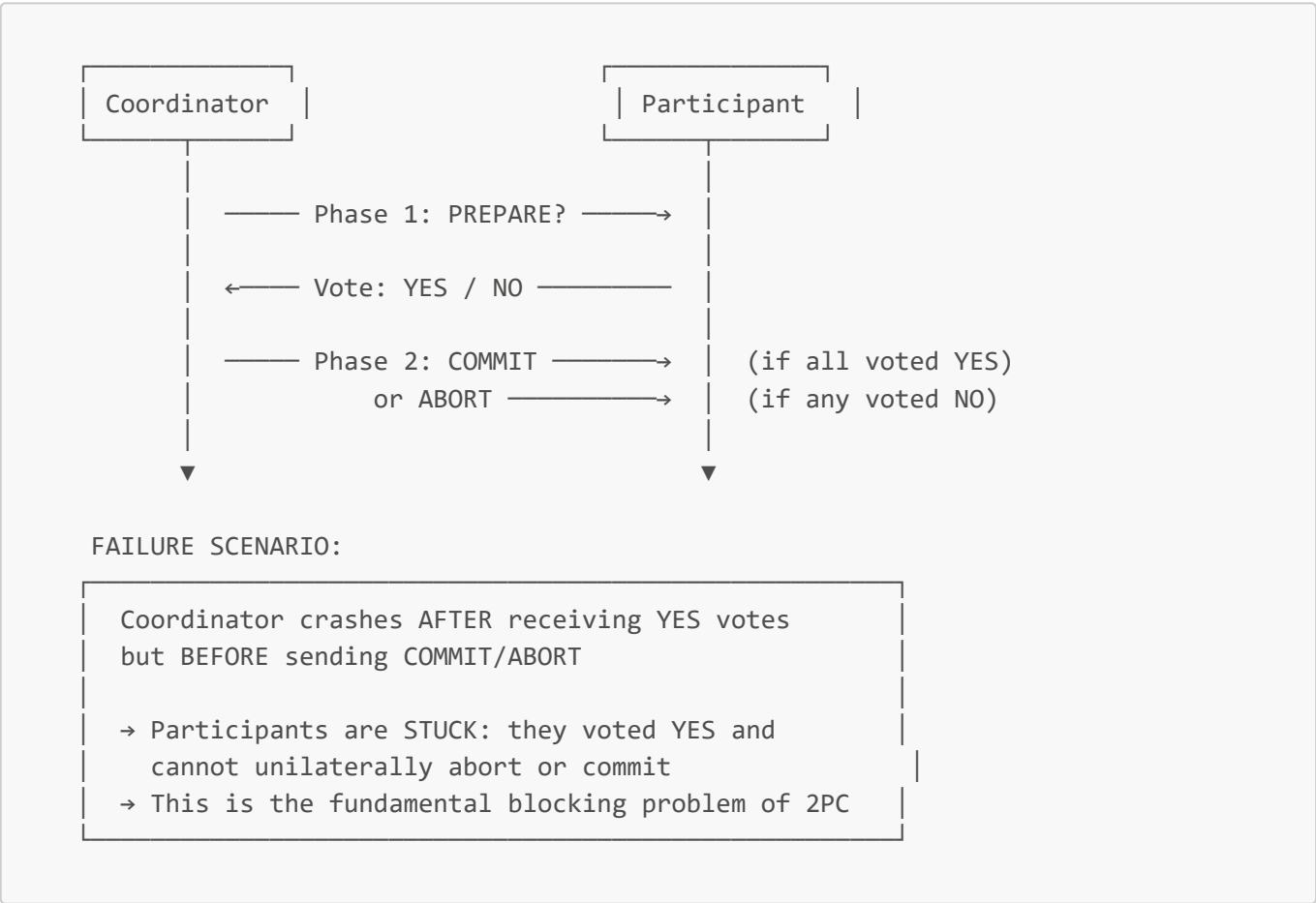
There's a meaningful gap between *single-machine correctness* and *multi-party correctness*. On one machine, ACID transactions give you strong local guarantees — atomicity, consistency, isolation, durability. But the moment a second machine enters the picture, everything gets harder:

- **Who decides if a transaction commits?** → You need a coordinator.
- **What if the coordinator crashes mid-decision?** → Participants might block forever.
- **What if a participant crashes after voting "yes"?** → The coordinator must handle partial agreement.

Two-Phase Commit (2PC) clarified this beautifully. It separates two concerns that are easy to conflate:

Concern	What It Means
Durability	The update is safely written to storage and survives a crash
Agreement	All parties have decided the same thing — commit or abort

2PC Timeline and Failure Anatomy



Takeaway: Strong guarantees come at the cost of extra round trips and new failure modes. This tradeoff echoes everywhere — in database replication, in microservice sagas, and in consensus protocols like Raft and Paxos.

4) APIs to Load Balancers: The Practical Face of Distribution

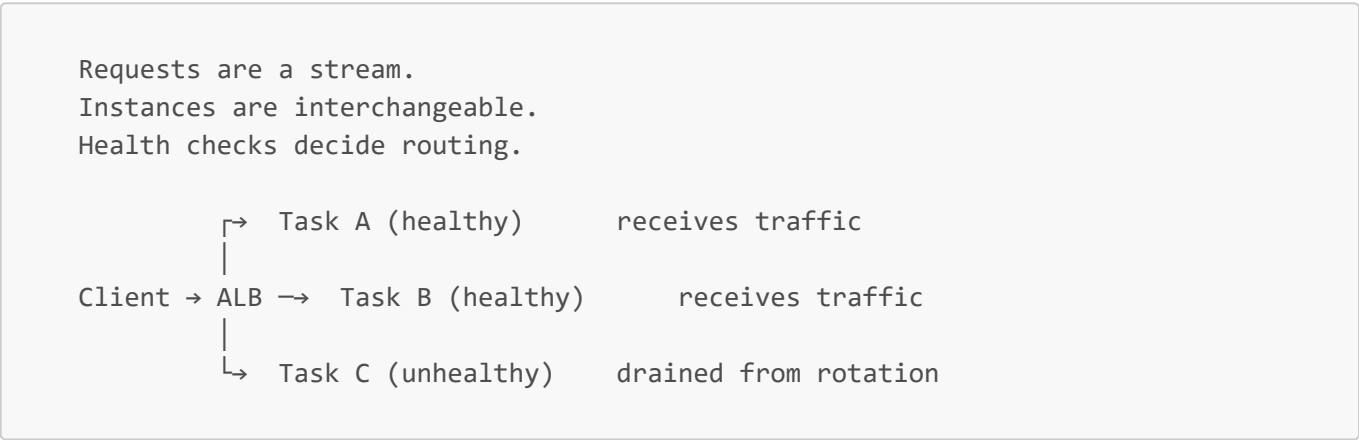
The Boundary Between Code and Infrastructure

Building REST endpoints in Go felt familiar — it's just programming. But the moment we moved off `localhost`, I crossed a boundary into infrastructure territory. Suddenly, the questions changed:

When Running Locally	When Running in Production
"Does my handler return the right JSON?"	"Which instance received this request?"
"Is my code correct?"	"Is my instance healthy?"
"How fast is my function?"	"What does the 99th percentile look like across all replicas?"

The Load Balancer Mental Model

Once we introduced load balancers, the mental model crystallized into something elegant:



This is a powerful abstraction: the client doesn't know (or care) which replica handles its request. The ALB continuously probes each target, and the moment one fails a health check, traffic is rerouted — no human intervention, no downtime for healthy instances.

What I Internalized

The load balancer isn't just a traffic splitter — it's the system's **first line of resilience**. Combined with multiple replicas, it transforms the failure model from *"one crash = total outage"* to *"one crash = slight capacity reduction."*

5) Reproducibility Through Automation: Terraform and Containers

Why This Matters More Than It Sounds

Early in the course, I set up infrastructure by clicking through the AWS console. It worked, but it was fragile — I couldn't reliably reproduce it, explain it to someone else, or tear it down cleanly. Terraform and Docker changed that completely.

The Three Layers of Automation

Layer 3: ORCHESTRATION (ECS + Fargate) "Run N copies of this container, restart on failure, and register with the load balancer"
Layer 2: PACKAGING (Docker + ECR) "Bundle my Go binary + dependencies into a portable image and store it in a registry"
Layer 1: INFRASTRUCTURE (Terraform) "Declare the VPC, subnets, security groups, ALB, ECS cluster, and auto-scaling policies as code"

Key Contrasts That Made It Click

Manual Approach	Automated Approach
"I remember the 12 console steps"	<code>terraform apply</code> — done
"Works on my machine"	Docker image runs identically everywhere
"I think I deleted everything"	<code>terraform destroy</code> — verified cleanup
"Let me SSH in and deploy"	Push to ECR → ECS rolls out new tasks

The Habit I Valued Most

Lifecycle thinking: always destroy resources when done, track cloud spend, and treat infrastructure like code you version, review, and rerun. This discipline prevents the silent budget drain that plagues cloud learners.

6) Performance: Percentiles, Tail Latency, and Bottleneck Identification

Why Averages Lie

The most satisfying experiments were the ones where **the graphs told the story**. HW1b and later load-testing exercises pushed me past the trap of average latency. Consider this scenario:

Request latencies (ms): 12, 11, 14, 13, 12, 11, 310, 12, 13, 11

Mean: ~42 ms ← looks bad, but misleading

Median: 12 ms ← looks fine

p95: 310 ms ← THIS is what the unlucky user feels

p99: 310 ms ← and it might be even worse at scale

A small fraction of slow requests can **dominate perceived quality**. This is the long-tail distribution, and it's what users experience as *"sometimes it's slow."*

Bottleneck Diagnosis in Practice (HW6 Part II)

Load testing with **Locust** against the product search service revealed a clean bottleneck signature:

Observation During Load Test:

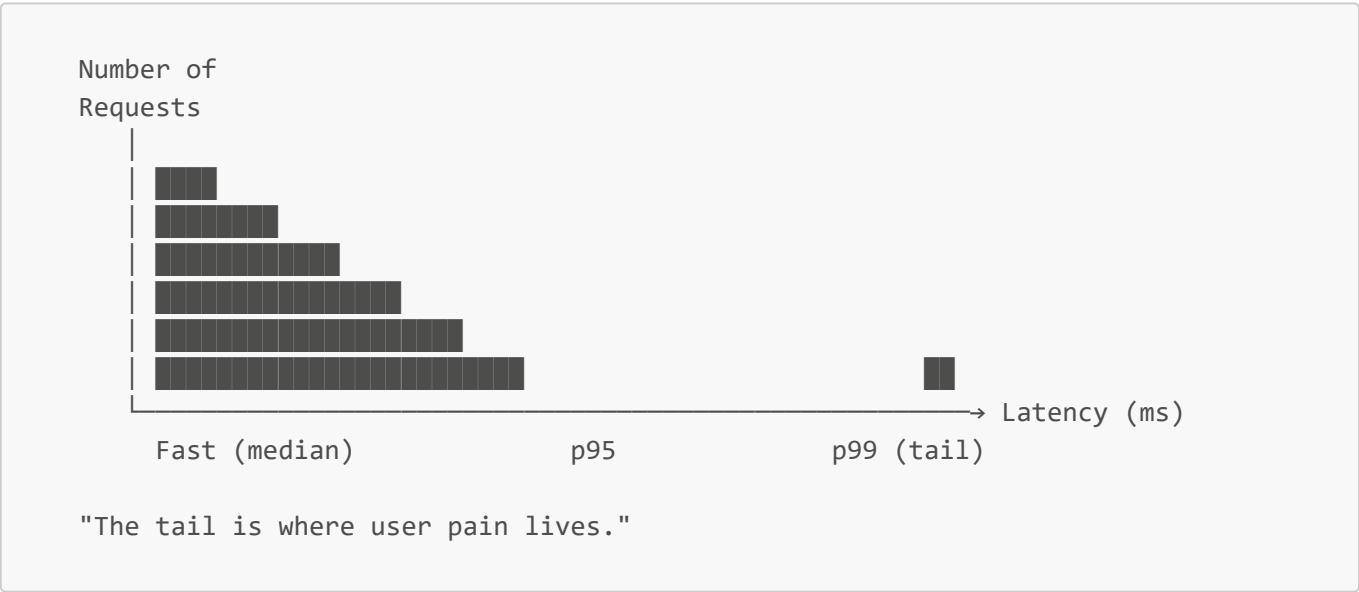
CPU Utilization	95%+	← saturated
Memory Usage	~35%	← plenty of headroom
Network I/O	~25%	← not the bottleneck

Diagnosis: CPU-bound workload (checking 100 products per request)
Implication: Adding more memory or network bandwidth won't help
Solution: Scale horizontally (more tasks) or vertically (bigger CPU)

HttpUser vs FastHttpUser

An interesting subtlety: Locust's default **HttpUser** adds client-side overhead that can skew results. Switching to **FastHttpUser** removes that noise — but it **only matters when the server isn't already the bottleneck**. If the server is saturated, client overhead is irrelevant.

Latency Distribution Mental Model



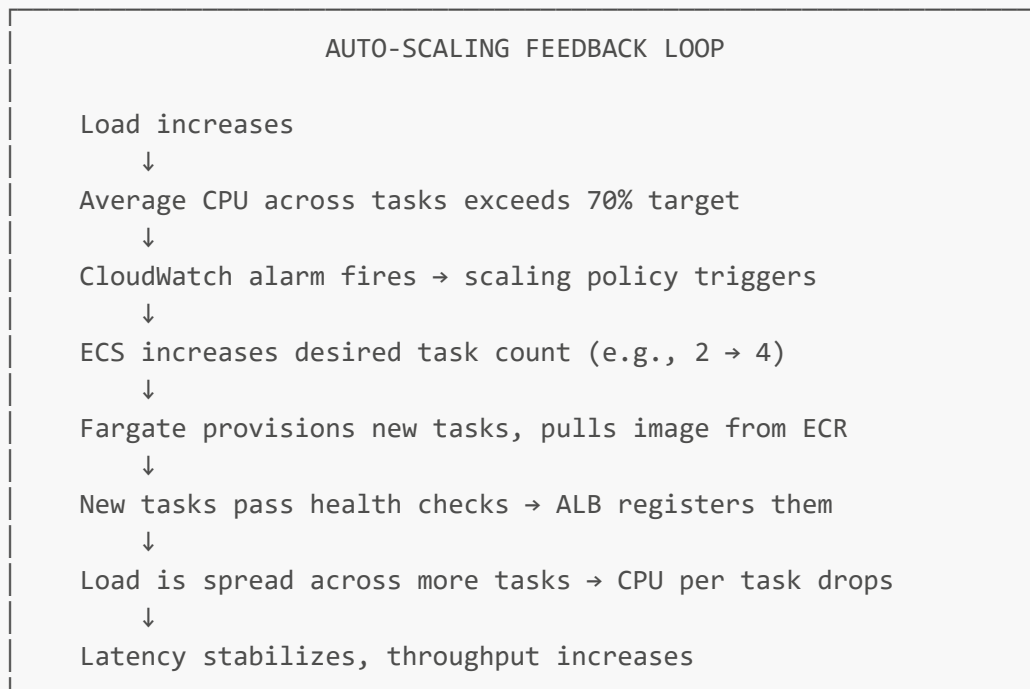
7) Horizontal Scaling with ALB and Auto-Scaling: Resilience Becomes Visible

The Turning Point

HW6 was where everything came together. With a fixed cost per request (scanning exactly 100 products), no algorithmic optimization existed — the work is inherently expensive. The question shifted from *"how do I make this faster?"* to *"how do I throw more compute at it safely?"*

The Auto-Scaling Feedback Loop

This is the architecture that made scaling feel like a **living system**:



What Made It Click: Killing a Task Under Load

The most memorable moment was **intentionally killing a task during a load test**. Instead of a catastrophic failure, the system degraded gracefully — the ALB stopped routing to the dead task, the remaining replicas absorbed the load, and auto-scaling eventually brought a replacement online. *This* is what resilience looks like beyond diagrams.

Horizontal vs Vertical Scaling Decision

When to scale VERTICALLY (bigger instance):

- ✓ Single-threaded bottleneck
- ✓ Need more memory per task
- ✓ Simpler operationally (no load balancer needed)

When to scale HORIZONTALLY (more instances):

- ✓ Workload is parallelizable across requests
- ✓ Need fault tolerance (one instance dying ≠ outage)
- ✓ Can scale beyond the limits of a single machine
- ✓ Auto-scaling can match demand dynamically

8) Readings That Changed How I Think

Lamport — Time, Clocks, and the Ordering of Events

You cannot assume global time in a distributed system. Lamport gave me the language to reason about **causality** without relying on perfectly synchronized clocks. The "happens-before" relation is a tool for

answering: "Can event A have influenced event B?" — and if you can't establish that, you must treat them as concurrent.

This complements every practical lab: when you add replicas, your system becomes concurrent **across machines**, and you need ordering guarantees for correctness and debugging.

Parnas — On the Criteria for Decomposing Systems into Modules

Parnas's key idea — **hide design decisions behind stable interfaces** — mapped directly to microservices and bulkheads. Decomposing systems by information hiding (not by flowchart steps) reduces coupling and enables independent evolution. This is why you can update one microservice without redeploying the entire system.

Mixu — Distributed Systems for Fun and Profit (Ch. 1–2)

Mixu provided a clean framework for *why the hard parts exist*: partial failure is unavoidable, networks are unreliable, and coordination has real costs. Distribution isn't magic — it's a set of tradeoffs you navigate with eyes open.

Interview-Ready Takeaways

These are the principles I would bring to a systems design interview, each grounded in something I built or measured during the course:

#	Principle	Grounded In
1	Correctness first, then performance	Race detector, locks, and clear invariants before any optimization pass
2	Measure, don't guess	Percentiles, CloudWatch resource graphs, and controlled experiments reveal the real bottleneck
3	Prefer reproducible infrastructure	Terraform + Docker + ECR/ECS reduce human error and make iteration fast
4	Design for failure	Load balancers, health checks, and multi-replica services make outages survivable
5	Good boundaries scale teams	Modular design (Parnas) reappears in microservice architecture — clear interfaces reduce ripple effects

Closing Reflection

The topics I enjoyed most were the ones that forced me to **collect evidence and defend conclusions**: concurrency experiments where the race detector proved my code was broken, load tests where percentile charts revealed hidden bottlenecks, and scaling exercises where I watched auto-scaling respond to real traffic.

The throughline of CS 6650 is that distributed systems thinking is a *discipline of measurement and reasoning under uncertainty*. You don't memorize architectures — you learn to ask the right questions, run the right experiments, and interpret the results honestly.

"If you can't measure it, you can't improve it."

"If you can't reproduce it, you can't trust it."

"If you can't break it gracefully, you can't run it in production."